

OrbitOS

CRM + ERP Agent

技术方案与经验分享

Agent 架构怎么设计、上下文与记忆怎么管、为什么没有 RAG、数据不用数据库怎么存,前端每一个依赖为什么选它——以及过程中真实翻的车。

项目	OrbitOS — 面向 B2B 制造业的 CRM + ERP Agent 演示系统
周期	2026-09-02 00:16 → 05:47(单日连续开发,5 小时 31 分)
规模	31 个 commit / 4,974 行源码 / 1,229 行测试 / 141 条用例
模型	智谱 GLM-4.5-Flash(免费档) · 原生 function calling
部署	Cloudflare Pages + Pages Function / 国内可直连

序 全景与导航

先给全貌,再逐层展开。

15	3	0	0
Agent 工具	权限 拦截 层	数据 库	向量 库 / RAG
2	12	2	6.4s
模型 调用 阶段	执行 环轮 次上 限	跨轮 记忆 轮数	单次 调用 实测 延迟

这是什么

OrbitOS 是一个「会自己干活的 CRM+ERP」:用户用自然语言提出业务问题,系统自主拆解成执行计划、依次调用 15 个业务工具查数据,写操作前弹确认卡请人批准,批准后真实修改数据, 首页看板与风险卡实时联动。系统内置四个角色,每个角色能调用的工具、能看到的数据行、能看到的字段都不同,且每一次过滤都会明说自己过滤了什么。

贯穿全项目的产品原则:不要静默过滤。 系统因为权限、因为筛选、因为并发而少给用户看了东西、少做了一件事,就必须当场说清楚「少了什么、为什么少」。这条原则从一个文案要求,最后演化成了权限设计、工具返回结构、提示词硬规则三处的共同约束。

先回答四个最常被追问的问题

问题	答案	一句话说明
是 ReAct 架构吗?	不完全是	Plan-and-Execute(先出计划)+ ReAct 式执行环,但用原生 function calling 而非 Thought/Action 文本协议,且思考链主动关闭。详见 § 4.2
有 RAG 吗?	完全没有	零向量库、零 embedding。业务知识只经两条路进模型:提示词里的工具目录、工具返回的 JSON。详见 § 4.5
有数据库吗?	完全没有	一个内存 DbSnapshot 对象,由固定种子生成,放在 Zustand 里。详见 § 3
记忆存在哪?	三层,均在客户端	单次运行内的 messages / 跨轮只带最近 2 轮 / localStorage 持久化。无服务端存储。详见 § 4.4

目录

一	项目全貌与角色边界	1 页
二	前端技术栈:每一个是什么、为什么选它	逐项 12 条

三	数据层:没有数据库,数据怎么解决	含代价
四	Agent 设计详解	全文重点 · 含 4 段源码
五	其余产品决策	决策 / 理由 / 代价
六	Bad Case:7 个真实缺陷	现象 / 根因 / 处置 / 教训
七	做得不好的地方	含我的 3 个判断错误
八	改进方向	Agent / 产品 / 工程
九	可复用的经验清单	15 条

一 项目全貌与角色边界

后面所有技术决策的坐标系。

1.1 产品命题

对标 Monday CRM 的信息架构,但要回答一个 Monday 没回答的问题: **当 CRM 里装进一个能自己调工具、自己改数据的 Agent,权限体系应该长什么样?** 所有功能都是为把这个命题演示清楚而存在的。

业务场景选的是 **B2B 制造业的销售—供应链协同**:销售签了单、交期承诺出去了,但库存不够、采购在途、供应商交期不一。这是一个天然需要跨模块查数据、需要写操作、且不同岗位看到的東西天然不同的场景——三个条件缺一个,权限演示就没有意义。

1.2 四个角色与它们的边界

角色	数据范围	可用工具	特殊边界
销售代表 · 张伟 U-001	本人名下客户 / 订单 / 商机	9 / 15	应收、采购单 整类无权
销售总监 · 李娜 U-004	本团队全部数据 + 全公司汇总指标	11 / 15	采购单整类无权
供应链主管 · 王强 U-006	全公司库存 / 采购 / 供应商 / 订单 / 客户	10 / 15	订单金额与单价 脱敏为 *** ;商机、应收整类无权
CEO · 陈立 U-008	全公司全量	15 / 15	可执行任何写操作,但会收到 越权代办提示 并留痕

注意「整类无权」和「有范围、范围外还有」是两件不同的事,它们在代码里是两个字段、在界面上是两套文案。这个区分是 § 4.9 防幻觉设计的基础。

二 前端技术栈:每一个是什么、为什么选它

全部依赖列在下面,没有隐藏项。生产依赖 8 个,开发依赖 13 个,没有 UI 组件库。

2.1 React 19.2 — UI 框架

是什么:组件化的界面库,把界面拆成可复用的函数。**为什么:**生态最大,Recharts、React Router 这类库都是围绕它做的; React 18 引入 `useSyncExternalStore` 之后,外部状态库的接入成本降到几乎为零,这直接影响了 2.5 的选型。**没选 Vue/Svelte** 是因为图表和路由生态在 React 侧更成熟,而这个项目的核心工作量在数据和 Agent,不该花在找替代库上。

2.2 TypeScript 6.0 — 类型系统

是什么:给 JavaScript 加静态类型,编译期发现错误。**为什么:**这个项目里它是**第一道测试**。四个角色、五类实体、15 个工具的参数,全部用联合类型和 `Record<Role, T>` 表达——漏写一个角色分支,编译直接失败。`rbac.ts` 里每个 `scopeXxx` 都是对 `user.role` 的穷尽 switch,新增角色时编译器会逐个报错提醒你补上,这比任何文档都可靠。构建脚本是 `tsc -b && vite build`,类型不过关直接构建失败。

2.3 Vite 8.2 — 构建工具

是什么:开发服务器 + 打包器。**为什么:**冷启动秒级,改一行代码热更新几乎无感;产物是**纯静态文件**,可以直接扔到任何 CDN——这一点直接决定了部署方案能不能免费。**没选 Next.js:**这个项目没有服务端渲染需求,也没有服务端数据源,上 Next 等于为了一个不存在的问题引入一整套框架约束。

2.4 Tailwind CSS 3.4 — 样式方案

是什么:原子化 CSS,直接在 `className` 里写 `px-4 text-sm` 这类工具类。**为什么:**不需要维护 CSS 文件和类名体系;Monday 的品牌色写进 `tailwind.config.js` 一处(`brand #0073ea / ok #00c875 / warn #fdb3d / danger #e2445c`),全站统一。**没上 v4:**v4 改用了新的 PostCSS 链路,当时对 Vite 8 的兼容还不稳定,演示项目不值得赌。

2.5 Zustand 5 — 状态管理

是什么:极简的全局状态库,三行代码建一个 store。**为什么:**整个业务数据库就是这个 store 里的一个字段(见 § 3),不需要 Redux 的 action/reducer/middleware 那一整套仪式。**代价(真实的):**项目里的订阅全是无选择器的整店订阅,意味着 store 一变,所有订阅组件都重渲染。这直接导致了一个设计决定——Agent 会话的临时状态**刻意留在组件内部,不进 store**:一次问询要触发几十个事件,若放进全局,每张卡片都会重渲染整棵应用树。

2.6 React Router 7.18 — 路由

是什么:把 URL 映射到页面组件。**为什么:**八个业务页面需要各自的地址,演示时才能直接用链接过去。只用了最基础的声明式路由,**没用 v7 的 loader/action 数据层**——那套是为有后端的应用设计的,这里数据全在内存里,用它反而绕远。

2.7 Recharts 3.10 — 图表

是什么:基于 SVG 的 React 图表库。**为什么:**选它几乎是被迫的——它是**当时唯一原生兼容 React 19 的成熟图表库**。另外它的 API 是声明式的(`<BarChart><Bar /></BarChart>`),和 React 心智一致,不需要在 `useEffect` 里手动管理图表实例的生命周期。**没选 ECharts:**功能更强,但它是命令式 API,要自己处理挂载/销毁/尺寸变化,在这个项目里是净负担。

2.8 lucide-react 1.39 — 图标

是什么:开源图标库的 React 版本,每个图标是一个组件。**为什么:**按组件引入,树摇之后只打包实际用到的那十几个图标,不像图标字体那样整包几百 KB。风格线性、克制,和 Monday 的视觉语言接近。

2.9 clsx + tailwind-merge — 类名合并

是什么:两个加起来约 3KB 的小工具。clsx 负责条件拼接类名, tailwind-merge 负责解决 Tailwind 类冲突。**为什么:**写 `cn('px-2', isActive && 'px-4')` 时, 不加 tailwind-merge 会同时输出两个 padding 类,谁生效取决于 CSS 顺序——这是 Tailwind 项目里最常见的隐蔽 bug。它按 Tailwind 的语义规则保留后者。

2.10 Vitest 4.1 + oxlint — 测试与静态检查

是什么:Vitest 是测试运行器,oxlint 是 Rust 写的 linter。**为什么:**Vitest 直接复用 Vite 的配置和转译链路,零额外配置就能测 TypeScript; oxlint 比 ESLint 快一个数量级,在这种需要频繁跑检查的高强度开发节奏里差别明显。项目共 **12 个测试文件、1,229 行、141 条用例**,全部是纯函数级——这个取舍的代价在 § 7.1 里单独说。

2.11 Cloudflare Pages + Pages Function — 部署

是什么:静态站托管 + 一个边缘函数。**为什么:**三个硬需求同时满足:免费、国内可直连、支持服务端代码。最后一点是关键——**模型 API Key 绝不能进前端包**,所以需要有一个中转。整个后端只有 20 行:

全部后端代码 · `functions/api/chat.ts`(20 行)

```
interface Env { ZHIPU_API_KEY: string }

export const onRequestPost: PagesFunction<Env> = async (ctx) => {
  const key = ctx.env.ZHIPU_API_KEY // ← Key 只存在于 CF 环境变量
  if (!key) {
    return new Response(JSON.stringify({ error: 'NO_KEY' }), { status: 503, ... })
  }
  const body = await ctx.request.text()
  const upstream = await fetch('https://open.bigmodel.cn/api/paas/v4/chat/completions', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${key}` },
    body, // ← 请求体原样透传, 不做任何加工
  })
  return new Response(upstream.body, { status: upstream.status, ... })
}
```

它是一个纯粹的鉴权代理:不解析请求体、不缓存、不改写。所有 Agent 逻辑都在浏览器里跑。

2.12 刻意没用的东西

没用	理由
UI 组件库 (Antd / MUI / shadcn)	目标是复刻 Monday 的视觉语言,组件库的默认样式反而要花力气对抗。项目里的表格、状态芯片、卡片都是手写的,总共不到 400 行
React Query / SWR	它们解决的是「服务端数据的缓存与同步」。这个项目没有服务端数据——数据就在内存里,读它是同步的
状态机库 (XState)	Agent 执行环确实是个状态机,但只有 12 轮上限的一个 for 循环,引入状态机库是过度设计
i18n	演示只面向中文用户
任何 Agent 框架 (LangChain / LlamaIndex)	这一条最重要: 整个 Agent 环加起来 157 行(见 § 4.3)。框架会带来一层不透明的抽象,而这个项目的卖点恰恰是「权限如何贯穿每一次工具调用」——那必须是我自己能逐行解释的代码

三 数据层:没有数据库,数据怎么解决

这一节回答「你的数据存在哪」——答案可能和预期不同。

3.1 答案:一个数据库都没有

没有 MySQL,没有 PostgreSQL,没有 SQLite,没有 IndexedDB,没有任何后端存储。整个业务数据库是内存里的一个 **JavaScript 对象**,类型叫 `DbSnapshot`,在应用启动时由 `generateSeed(42)` 生成一次,放进 Zustand store 的 `db` 字段。所有工具的「查询」,本质是对这个对象里的数组做 `filter` / `reduce`。

3.2 数据从哪来:确定性伪随机 + 显式埋雷

种子生成器用的是 `mulberry32(42)`——一个 32 位确定性伪随机算法。同样的种子永远产出同样的数据。生成规模:

users	customers	products	suppliers	orders	opportunities	receivables	purchaseOrders
12	48	60	8	160	90	120	55

生成之后再叠一层 `applyPlantedScenario`——**显式埋雷**:制造三张交期紧张的订单(相对当天 +6/+8/+9 天),其中两张真的缺货,缺口合计 48 台;把唯一能供该 SKU 的供应商设成交期 7 天;并且确保在途采购单里**不含**缺货的那个 SKU。

埋雷的目的是造出一条完整因果闭环:发现风险 → 定位缺口 → 比选供应商 → 下单(请人批准)→ 库存补上 → 风险消失 → 首页 KPI 归零。没有埋雷,Agent 就只是个查询机器人。

3.3 写操作怎么工作:整库克隆 + 引用替换

每一次写入(下采购单、改交期、锁库存、建任务)走同一条路径:

写入路径 · `src/lib/store.ts`(节选)

```
applyMutation: (m) => set(s => {
  const db = structuredClone(s.db) // ← 深拷贝整个数据库
  switch (m.kind) {
    case 'createPurchaseOrder':
      db.purchaseOrders.unshift(m.po)
      // 加急采购视为可用量即时补足 (演示语义: 锁定供应)
      for (const l of m.po.items) {
        const inv = db.inventory.find(i => i.skuId === l.skuId)
        if (inv) { inv.onHand += l.qty; inv.available = inv.onHand - inv.reserved }
      }
      break
    ...
  }
  return { db } // ← 换掉引用, React 据此重渲染
})
```

为什么整库克隆而不是原地改:不可变快照让「有没有变」退化成一次引用比较。React 和 Zustand 都靠引用是否相同来决定要不要重渲染;如果原地改,引用没变,界面不会更新——那就得手动通知每一个订阅者。这也是演示里「批准采购单后风险卡当场消失」能自动生效的原因:数据一换引用,首页的风险计算重跑,卡片自己就没了。

3.4 为什么不上数据库

1 演示必须可重复

刷新即回到初始状态,意味着这套演示可以无限次重跑。如果数据落盘,上一次把采购单批了,下一次风险卡就不存在了,演示的高潮直接没有。这是决定性理由。

2 零后端才能同时满足免费 + 国内可直连

纯静态产物可以扔到任何 CDN。一旦引入数据库,就需要一台常驻服务器或托管数据库服务,免费额度、国内访问、冷启动延迟三个问题会同时出现。

3 没有数据隐私面

所有数据都是自造的、公开的、每个访问者拿到的都一样。不存在「A 用户的数据泄漏给 B 用户」这类风险——因为根本没有跨用户的数据存储。这让演示可以放心公开。

4 权限逻辑因此可以做成纯函数

这是最被低估的一条。`scopeOrders(db, user)` 是一个纯函数: 给定数据和用户,返回可见的行。没有数据库,就没有 SQL 拼接、没有连接池、没有异步。于是**所有权限规则都能被单元测试直接钉死**——项目里 141 条测试有相当一部分是在断言「张伟能看到几条、看不到哪几条」。如果权限是写在 SQL 的 WHERE 里,这些测试要么写不出来,要么要起一个真数据库。

3.5 代价(诚实说)

代价	说明
不能多人协作	每个访问者是一份独立的内存副本,互相看不见
不能跨设备	手机上的操作不会同步到电脑
刷新即丢	对演示是特性,对真实产品是致命缺陷
整库克隆是 $O(n)$ 写放大	每次写入都深拷贝全部约 550 条记录。这个量级下毫秒级完成、感知不到; 但这个方案在 50 万条时会直接崩掉 ,它成立的前提就是数据量小
没有事务、没有并发控制	单用户单线程,不需要;但这意味着这套代码一行都不能直接搬进生产

3.6 确实落盘的两样东西

业务数据不持久化,但有两个刻意的例外,都在 localStorage:

key	存什么	为什么它是例外
orbitos.ui.sidekick	抽屉展开 / 宽窄	界面偏好属于「用户的」而不是「演示的」。每次刷新都要重新展开抽屉是纯粹的骚扰
orbitos.sidekick.conversations	完整会话(含所有卡片与最近两轮问答)	问了三个问题后刷新一下就全没了,是无法接受的体验损失

两个读写模块都写成接受可选 `store` 参数的纯函数,每个 `getItem` / `setItem` 各自 try/catch,内容损坏时逐级回落,绝不抛异常。原因有二:一是 Safari 隐私模式下 localStorage 会直接抛错,不能让它把整个应用带崩;二是接受 `store` 参数意味着测试时可以传一个内存 Map 进去,在没有 jsdom 的环境里也能完整测试(见 § 7.1)。

3.7 一个踩过的存储坑:上限必须按角色分组

最初的会话上限是全局的 `slice(0, 20)`。后果是:演示时来回切四个角色,每个角色各问几句,某个角色的**全部历史会被其他角色的会话挤出存储**——用户看到的现象就是「我明明问过,记录不见了」。修法是把上限改成按 `userId` 分组:每角色 20 条活跃 + 30 条归档,归档的不占活跃名额。**教训:任何「保留最近 N 条」的策略,先问一句「N 是按谁算的」。**

四 Agent 设计详解

本文重点。整个 Agent 环是 157 行代码,没有用任何框架,下面逐层拆开。

4.1 整体架构



图 1 · 全景架构。业务数据一次都没有离开浏览器——离开的只有提示词和工具返回值。

4.2 是 ReAct 吗?——三种范式的取舍

范式	怎么工作	为什么没用 / 用了哪部分
单轮函数调用	一次调用,模型直接选一个工具,拿到结果就回答	没用。体验是黑盒:用户盯着转圈,几十秒后蹦出结论,中间发生了什么完全不可见,也无法中途质疑
ReAct	Thought → Action → Observation 的文本循环,模型边想边做,没有预先计划	用了它的执行环形状,没用它的协议。ReAct 靠文本格式约定来解析动作,容易解析失败;而 GLM 支持原生 function calling,由服务端保证结构化输出,更可靠。而且 ReAct 的推理是即时产生的,用户在动手前看不到全貌
Plan-and-Execute (本项目)	第一次调用先产出 JSON 执行计划,第二阶段按计划循环执行工具	主体采用。用户在第一秒就看到「它打算干五件事」,然后一件件亮起。这把「等待」变成了「围观」,也让推理过程可被质疑——计划不对可以当场打断,不用等它干完

准确的描述是:Plan-and-Execute 的骨架 + ReAct 式的执行环 + 原生 function calling 的协议 + 一个轻量的动态重规划出口。

一个需要坦白的取舍:思考链是主动关掉的(`thinking: {type:'disabled'}`)。这意味着没有显式的推理轨迹——模型的「想」发生在哪里、怎么从缺口推到供应商,在这个系统里是看不到的。用户看到的是计划(规划阶段的产物)和工具调用序列,而不是逐步推理。这是拿可解释性换延迟:开着思考链单次调用 26 秒,一个五步计划累计超过两分钟,演示节奏直接崩掉。如果这个产品要进真实企业,这个取舍要重新做——那时该保留推理轨迹并折叠展示,而不是丢弃。

4.3 上下文管理:两次调用分别塞了什么

一次问询会向模型发起 2 到 13 次请求:1 次规划 + 1~12 次执行。两个阶段的上下文构造完全不同。

阶段一 · 规划器(1 次调用,JSON 模式)

消息	内容
system	角色身份(姓名 + 岗位标签) · 权限描述文字 · 当前日期 <code>TODAY</code> · 最近对话块(见 § 4.4) · 本角色可用的工具目录(名字 + 一句描述) · 5 条拆解规则 · 输出 JSON 的格式模板
user	用户的原始问题

关键细节:工具目录是按角色过滤后才写进提示词的,并且明说「列表之外的工具不存在,不要规划任何依赖它们的步骤」。这是权限的第一层,也是让计划不出现「你无权做的事」的关键。规划器还被要求:如果问题超出角色权限,输出空 steps 并在 goal 里说明谁能做——这样越权请求得到的是一句解释,而不是一个跑不通的计划。

阶段二 · 执行环(最多 12 次调用,带 tools)

执行环主循环 · `src/agent/loop.ts`(节选,省略了事件发射与步骤推进)

```
const messages: ChatMessage[] = [
  { role: 'system', content: executorPrompt(o.user, plan, o.history) },
  { role: 'user', content: o.question },
]
const tools = toolSchemasFor(o.user.role) // ← 权限第一层: 只给这个角色的工具

for (let turn = 0; turn < MAX_TURNS; turn++) { // MAX_TURNS = 12, 熔断
  const res = await chat({ messages, tools })
  messages.push(res) // ← 上下文单调增长

  const calls = res.tool_calls ?? []
  if (!calls.length) {
    // 实盘复现: 计划含写入步骤时, 模型常用「建议你去采购 X 个」这样的文字收尾,
    // 而不去真的调用 create_purchase_order, 导致确认卡与后续联动全部不发生。
    const planHasWriteStep = plan.steps.some(
      s => s.expectedTools?.some(t => writeToolNames.has(t))
    )
    if (!nudged && !writeResolved && planHasWriteStep) {
      nudged = true // ← 只补推一次, 防死循环
      messages.push({ role: 'user', content: '计划中还有尚未执行的写入步骤。请直接调用'
        + '对应的写入工具去完成它, 不要只用文字给建议。...' })
      continue
    }
  }
  const refs = [...text.matchAll(/\[\[([^\]]+)\]\]/g)].map(m => m[1])
  o.emit({ type: 'final', text, refs: [...new Set(refs)] })
  return
}

for (const call of calls) {
  // 写工具 → 弹确认卡 → await 用户批准 (见 §4.8)
  const r = executeTool(name, args, ctx()) // ← 权限第二层 + 第三层
  messages.push({ role: 'tool', tool_call_id: call.id, content: JSON.stringify(r.result) })
}
```

上下文怎么增长,以及它的缺陷

每一轮循环向 `messages` 追加两类内容:模型的回复(含 `tool_calls`), 和**每个工具返回值的完整 JSON 序列化**。这个数组只增不减。

诚实的缺陷:这个系统没有任何上下文压缩、摘要或裁剪机制。 控制上下文体积靠的是一个**约定**而不是机制—— 每个工具自己返回精简结构(默认 `limit ?? 20`、只带必要字段)。一旦某个工具返回了大结果,上下文就会线性膨胀,而系统对此毫无防御, 只有 `MAX_TURNS = 12` 这个粗糙的熔断兜底。这在演示规模下够用(实测单次问询总 token 在几千量级), 但它是这个 Agent 设计里**最不能规模化的一处**。改进方向见 § 8.1。

其余的上下文控制手段:每次请求 45 秒超时(`AbortController`), `temperature` 固定 0.2 降低随机性, 以及 `usageLog` 记录每次调用的 token 用量—— 演示时可以现场报「这次问询花了多少 token」。

4.4 记忆:三层,分别存在哪、活多久

层	存什么	存在哪	活多久	上限
工作记忆	完整 <code>messages</code> 数组:系统提示、问题、模型回复、 全部工具返回值	函数内的局部变量	一次问询	12 轮循环
会话记忆	只有 <code>{ q, a }</code> —— 问题与最终答案。 工具中间过程不进	会话对象的 <code>history</code>	跨轮问答	只带最近 2 轮 ,每条答案截断到 600 字
持久记忆	完整会话:标题、创建者、全部卡片、 <code>history</code>	<code>localStorage</code>	跨刷新	每角色 20 条活跃 + 30 条归档

为什么只带最近 2 轮

这是免费档 token 压力和追问深度之间的实测取舍。观察真实使用: 演示场景里的追问深度基本不超过两轮(「采用方案 1」「那个订单交期改到什么时候」), 再往前的上下文对当前问题几乎没有贡献,却要一直占着 prompt token。

为什么工具中间过程不进跨轮记忆

两个理由。第一,太长——一次问询的工具返回加起来可能有几千 token, 带两轮就是上万,免费档撑不住。第二,**模型不需要它**: 下一轮问题若要用到同样的数据,它会重新调用工具拿到**最新**的值; 而旧的工具结果可能已经因为写操作而过期了。**带着过期数据回答,比重查一次更危险。**

这个系统「没有」的记忆

- **没有长期用户画像**—— 系统不会记住「张伟习惯先看逾期应收」这类偏好
- **没有跨会话记忆**—— 新开一个会话,前一个会话的内容完全不可见
- **没有服务端存储**—— 所有记忆都在用户自己的浏览器里,换台设备就没了
- **没有向量化的语义记忆**—— 不能问「上个月我们聊过的那个供应商」

这些缺失对演示项目是合理的范围划定,但如果要做成真实产品, **记忆层是第一个需要重写的模块**—— 它需要服务端存储,需要跨设备同步, 而一旦有了服务端存储,权限模型就要从「客户端过滤」升级为「服务端强制」。

4.5 知识库与 RAG:为什么一行都没有

这个项目没有知识库,没有 RAG,没有 embedding,没有向量数据库,没有文档切分。 模型获得业务知识只有两条路径:

- 1 提示词里的工具目录
- 规划阶段把该角色可用的工具名 + 一句话描述写进 system prompt。 这告诉模型「你能做什么」,但不含任何具体业务数据。
- 2 工具返回的 JSON
- 执行阶段模型调用工具,拿回结构化结果。这是**唯一**的业务数据来源,且提示词里有一条硬规则:「回答中出现的每一个数字、单号、客户名、日期,都必须来自工具返回结果」。

为什么不需要 RAG:检索的性质不一样

RAG 解决的是**语义检索**问题:用户的问法和文档的写法不一样, 需要靠向量相似度把相关段落找出来。而这个系统里的查询长这样:

「张伟名下逾期超过 30 天的应收有哪些」

→ `scopeReceivables(db, user).filter(r => r.status === '已逾期' && overdueDays >= 30)`

这是 **filter + aggregate**,不是语义相似。 字段是结构化的,条件是精确的, 结果是可穷尽的。用向量检索去做这件事,不但更慢、更贵,而且**会返回近似匹配**—— 在财务数字上返回「差不多相关」的记录,是灾难而不是特性。 **选择结构化查询不是省事,是选对了工具。**

什么情况下这个项目才该上 RAG

如果需求变成下面这几类,RAG 就成了必需:

场景	为什么必须 RAG
「这个客户的合同里付款条款怎么写的」	合同是非结构化长文本,条款位置不固定,只能靠语义定位
「按公司 SOP,交期延误超过 5 天该走什么流程」	制度文档,知识在文字里而不在字段里
「历史上类似的缺货是怎么处理的」	需要跨历史工单做相似案例检索,这正是向量检索的强项

一个更重要的观点,也是企业 Agent 最常翻车的地方: 如果这个项目真的引入 RAG,**最难的不是检索质量,是权限**。 结构化查询的权限过滤是确定的—— `scopeReceivables` 一个函数就把边界划死了, 而且可以用单元测试钉住。但向量检索的权限过滤要在**检索层**做: 你必须保证销售代表的向量查询**永远不可能**召回其他团队的合同片段。做法通常是给每个 chunk 打权限标签并在检索时前置过滤(pre-filter), 而不是检索完再筛(post-filter)—— 后者会导致「召回 10 条、过滤剩 2 条」, 结果质量塌陷,而且一旦有一处漏标就是静默泄漏,**没有任何编译器或类型系统能帮你发现**。 本项目 § 6.1 和 § 6.2 那两个越权缺陷,在结构化世界里尚且发生了两次; 换成 RAG,发现难度会高一个数量级。

4.6 工具设计:15 个工具怎么切分

工具名	组	可用角色	写?	说明
query_customers	CRM	全部 4 个	—	客户列表,供应链看不到财务字段
get_customer_detail	CRM	全部 4 个	—	客户详情
query_opportunities	CRM	销售代表 / 总监 / CEO	—	供应链整类无权
create_followup_task	CRM	销售代表 / 总监 / CEO	W	建跟进任务
query_sales_orders	ERP	全部 4 个	—	供应链可见但金额脱敏
get_order_detail	ERP	全部 4 个	—	订单详情
check_inventory	ERP	全部 4 个	—	库存查询
query_purchase_orders	ERP	供应链 / CEO	—	销售侧整类无权
get_supplier_options	ERP	供应链 / CEO	—	供应商比选(交期 / 准时率 / 价格系数)
create_purchase_order	ERP	供应链 / CEO	W	演示高潮:下加急采购单
query_receivables	分析	销售总监 / CEO	—	销售代表、供应链整类无权
aggregate_metrics	分析	销售总监 / CEO	—	4 种口径:营收 / 漏斗 / TOP 客户 / 状态分布
simulate_delivery_risk	分析	全部 4 个	—	本地确定性计算,不是模型估算
update_order_promise_date	写	销售代表 / 总监 / CEO	W	改承诺交期
reserve_inventory	写	供应链 / CEO	W	锁定库存

一个工具长什么样

工具定义 · `src/agent/tools/analytics.ts`(`query_receivables` ,节选)

```
{
  name: 'query_receivables',
  description: '查询应收账款。可按状态与最小逾期天数筛选，附客户名与逾期天数。',
  allowedRoles: ['sales_director', 'ceo'],    // ← 权限第一层：不在名单里的角色，
  isWrite: false,                            // ← 这个工具压根不会进发给模型的 tools 数组
  parameters: {                              // ← 标准 JSON Schema，模型据此填参
    type: 'object',
    properties: {
      status: { type: 'string', enum: ['未到期', '已逾期', '已回款'] },
      overdueDaysMin: { type: 'number', description: '最小逾期天数' },
      limit: { type: 'number' },
    },
  },
},
run: (a, ctx) => {
  const scope = scopeSummary(ctx.db, ctx.user, 'receivables') // ← 算出边界信息
  let rows = scopeReceivables(ctx.db, ctx.user)               // ← 权限第三层：过滤数据行
  if (a.status) rows = rows.filter(r => r.status === a.status)
  ...
  if (!filtered.length) return { found: false, reason: boundaryReason(scope, 'receivables') }
  return { scope, count: filtered.length, receivables }        // ← scope 随结果回给模型
},
}
```

每个工具都遵守同一份契约:声明可用角色 → 声明参数 schema → 在 run 里先算边界、再过滤数据、把 scope 连同结果一起返回。这份契约不是文档约定,是 TypeScript 的 ToolDef 接口强制的。

description 怎么写才能让模型选对工具

三条实践:(a) 写清楚能筛什么——「可按状态与最小逾期天数筛选」,比「查询应收」更能让模型知道该不该用它; (b) 枚举写进 schema 而不是描述里——`enum: ['未到期','已逾期','已回款']` 由服务端约束,比在描述里写「状态可以是…」可靠得多; (c) 消除歧义时要直说——`simulate_delivery_risk` 的描述结尾特意加了「这是本地确定性计算,不是估算」,防止模型把它当成一个可以自己推算的东西而跳过调用。

4.7 权限如何贯穿 Agent:三层拦截

L1	工具列表过滤	<code>toolsFor(role)</code> — 未授权的工具根本不会出现在发给模型的 <code>tools</code> 数组里。模型不知道它存在
L2	执行前角色复检	<code>executeTool()</code> — 真正执行前再查一次角色。防的是模型幻觉出一个它没见过的工具名
L3	数据行过滤	<code>scopeXxx(db, user)</code> — 工具内部按用户过滤数据行,再叠字段脱敏 <code>maskOrderForRole</code>

图 2 · 三层权限。任何一层单独存在都不够。

权限第二层 · src/agent/registry.ts

```
/** 权限第二层：执行前再查一次角色，防止 LLM 幻觉出一个它没有的工具名。 */
export function executeTool(name: string, args: any, ctx: ToolContext): ExecResult {
  const tool = ALL_TOOLS.find(t => t.name === name)
  if (!tool) return { ok: false, ms: 0, result: { error: `不存在名为 ${name} 的工具` } }
  if (!tool.allowedRoles.includes(ctx.role)) {
    return { ok: false, ms: 0, result: {
      error: 'PERMISSION_DENIED',
      reason: `当前角色无权调用 ${name}`,
      allowedRoles: tool.allowedRoles, // ← 告诉模型「谁能做」，好让它给出有用的解释
    } }
  }
  ...
}
```

为什么只做第一层不够：那等于把安全建立在「模型不会幻觉出一个它没见过的工具名」这个假设上，而这个假设不成立。第二层的价值是——即使模型凭空编出 `query_receivables`，销售代表的身份也会被挡下，并返回 `PERMISSION_DENIED` 和「谁能做」。这条路径有专门的负向测试钉死。

为什么还要第三层：工具能调 ≠ 数据能看全。`query_sales_orders` 四个角色都能调，但张伟拿到的是他名下的订单，王强拿到的是全公司订单但金额是 `***`。代码里有一条注释把这个原则写死了：

「本工具的 `allowedRoles` 已经只放供应链与 CEO 进来，两者都看全量，走 `scope` 是等价的。仍然走：数据层不该依赖工具层的门禁配置——改 `allowedRoles` 时数据边界要自动跟上。」

4.8 HITL 确认卡的完整生命周期

1	模型请求写操作	返回 <code>tool_calls: [create_purchase_order]</code>
2	拦截,不执行	检测到 <code>def.isWrite</code> ,不调用 <code>run</code> ,转而生成人类可读摘要 <code>confirmSummary(args, ctx)</code> :「将向锐驰机电采购 SKU-203 × 48 台,预计到货 09-09,金额 ¥591,360」
3	越权提示	若当前是 CEO 且该操作通常属于别的岗位,摘要前面追加「越权代办:该操作通常由供应链主管执行…操作会完整留痕」
4	挂起等人	发出 <code>confirm_request</code> 事件,UI 渲染确认卡, <code>await requestConfirm(...)</code> —— 整个 Agent 环在这里真的停住
5a	批准	执行 <code>executeTool</code> → 写入内存库 → 结果回灌 <code>messages</code> → 首页 KPI 与风险卡自动联动
5b	拒绝	把 <code>{ rejected: true, reason: '用户拒绝了该写操作, 请据此调整建议, 不要重复尝试' }</code> 当作工具结果回灌给模型
6	留痕	<code>pushAudit</code> 记录:时间、角色、用户、工具、参数、耗时、是否越权

图 3 · 写操作的六个阶段。第 4 步是这个产品能不能进企业的分水岭。

拒绝路径必须是真的,这是设计上最容易偷懒的地方。如果拒绝之后系统换个说法再弹一次,「你有否决权」就是一句空话。所以拒绝的回灌信息里明确写了「不要重复尝试」,并且 `writeResolved` 标志会置真——这意味着 § 4.10 的补推守卫不会因为用户拒绝而再推模型一次。批准和拒绝都算「这一步已有结论」。

4.9 防幻觉的四道防线

第一道:提示词硬规则

执行器 system prompt 里有 7 条编号规则,其中 4 条是直接针对幻觉的(以下为原文):

- 1.【禁止编造】回答中出现的每一个数字、单号、客户名、日期,都必须来自工具返回结果。绝不估算、推测或凭常识填充。宁可说"数据不足"也不要编。
- 3.【空结果】工具返回 `{"found": false}` 时,明确告诉用户"未找到相关数据"及原因,不要用其他数据替代。
- 4.【权限】工具返回 `PERMISSION_DENIED` 时,直接说明当前角色无权访问,并指出哪个角色可以,不要绕路猜测。

第二道: `[[ref]]` 溯源芯片

规则 2 要求引用具体记录时用双方括号标注,例如 `[[S0-2026-0412]]`。主循环用正则把它们抽出来,UI 渲染成可点击的芯片,点一下直接跳到那条数据。这是对「幻觉怎么防」这个必问题的产品化回答:与其论证模型不会编数字,不如让每一个数字都能被一键验证。可验证是比可信更强的承诺,因为它不需要对方信任你。

第三道:scope 范围披露(防「越界断言」)

这是本项目最有特色的一道。工具返回值里始终带一个 `scope` 字段,告诉模型「你看到的是多少条、总共多少条、按什么范围过滤的」,提示词里对应一条硬规则:

- 4a.【范围披露】...当 `scope.hidden > 0` 时,你的结论必须带范围限定语(例如「在你负责的范围内…」),并说明范围外还有多少条不可见。严禁使用「**全公司只有 / 一共只有 / 没有任何**」这类越界断言——你看到的是权限范围内的数据,不是全公司的全部数据。

没有这条,一个只能看 9 个客户的销售代表问「我们几个汽车零部件客户」,模型会理直气壮地回答「全公司共 2 个」——数字没编,结论是错的。这类幻觉比编造数字更危险,因为它看起来完全可信。

第四道:空结果的边界文案

「整类无权」和「有范围、范围外还有」必须是两套文案,不能共用模板:

显式边界文案 · `src/lib/rbac.ts`

```
export function boundaryReason(s: ScopeSummary, entity: ScopedEntity): string {
  const label = ENTITY_LABEL[entity]
  if (s.denied) // ← 整类无权
    return `你没有查看${label}的权限。全公司共 ${s.total} 条,均超出你的查看范围。`
  const base = `${s.basis}没有符合条件的${label}` // basis: 你本人名下 / 你在团队 / 全公司
  return s.hidden > 0 // ← 有范围,范围外还有
    ? `${base}; 全公司另有 ${s.hidden} 条,超出你的查看范围。`
    : `${base}。`
}
```

如果两者共用一套模板,销售代表查应收会得到「你没有查看应收的权限内没有记录」这种既拗口又错误的表达。

`denied` 是一个显式的白名单集合(`DENIED_PAIRS`),判据是「角色对该实体压根没有入口」,而不是「过滤后恰好为空」——这两者的区别是这段代码里最关键的设计。

4.10 补推守卫与动态重规划

补推守卫

线上首次实盘暴露的问题(详见 § 6.4):模型走完查询步骤后,用文字写「建议向锐驰机电采购 48 台」就收尾了,没有真的调用 `create_purchase_order`,于是确认卡不弹、数据不改、KPI 不动。

守卫的逻辑在 § 4.3 的代码里:当模型某一轮**没有返回任何工具调用**时,先不急着重收——检查计划里是否含写入步骤、是否一次都没执行过。若是,追加一条 user 消息把它推回去。**关键约束是只推一次** (`nudged` 标志),否则模型坚持不调用时会死循环。这条守卫有 5 条测试,断言的是「**模型被调用了几次**」而不是最终结果——因为要验证的正是这个控制流。

动态重规划

计划是执行前一次性产生的,但执行中难免发现计划外的步骤。出口设计得很轻:提示词规则 6 要求模型「先输出一句 `需要追加步骤:XXX`,再调用工具」,主循环用一个正则去匹配这句话,匹配到就往计划里追加一个步骤并发出 `plan_amended` 事件,界面计划清单当场多出一行。

这个设计的诚实评价:它很脆。靠正则匹配自然语言是不可靠的——模型换个说法就匹配不上。更稳的做法是提供一个 `amend_plan` 工具,让重规划走 function calling 的结构化通道。之所以没这么做,是因为把它做成工具会让模型有机会**滥用**它(每一步都追加一个步骤),而当时没有时间设计约束。这是一个「知道更好的做法但选择了够用的做法」的取舍,不是没想到。

4.11 降级:录播兜底

模型不可用时,自动切换到预录事件流重放,把两个核心剧本原样演一遍。这是对演示场景的风险对冲——免费模型 + 现场网络 = 高概率翻车。

但有一条重要的自我约束:**录播只覆盖两个剧本,问第三个问题时不猜、不硬凑**,直接诚实提示「录播仅内置两个场景」。理由是——断网时播出一段答非所问但看起来很真的内容,被当场识破的伤害,远大于承认降级。**已知未解决:**如果模型在执行到一半时挂掉,录播会从中途接管,真实卡片和重放卡片会叠在同一条会话里,出现两套计划清单。

4.12 成本与延迟:实测数据

项	数值	说明
关思考链前 · 单次调用	26 秒 / 489 输出 token	GLM-4.5 默认开推理链,绝大部分输出 token 花在 <code>reasoning_content</code> 上
关思考链后 · 单次调用	6.4 秒 / 34 输出 token	传 <code>thinking: {type:'disabled'}</code> ; <code>tool_calls</code> 照常返回,功能无损
一次问询总调用数	2 ~ 13 次	1 次规划 + 1~12 次执行
单次请求超时	45 秒	<code>AbortController</code>
并发上限	2 路	免费档限制。超出返回 HTTP 429(错误码 1302)
限流处置	退避 6 秒,重试 1 次	并发冲突是秒级的,一次重试往往就够;第二次仍失败才切录播
兜底重试	去掉 <code>thinking</code> 参数重试	防御性设计:若服务端哪天不认这个参数(非 429 的 4xx),去掉它重试。 宁可慢,不可挂

`usageLog` 记录每次成功调用的 prompt / completion / total token, `sumUsage()` 汇总。演示时可以现场报出「这次问询花了多少 token」——这对企业客户是个高频问题,能当场回答比事后估算有说服力得多。

五 其余产品决策

第四节没覆盖到的产品层取舍,仍按「决策—理由—代价」三段写。「代价」这一栏是刻意留的:一个说不出代价的决策,通常意味着还没想清楚它在跟什么做交换。

5.1 为什么是 CRM+ERP,而不是一个通用聊天 Agent	
决策	把 Agent 放进一个有明确岗位分工、有真实写操作、有跨模块因果链的业务系统里。
理由	通用 Agent 演示很难回答「凭什么信它」。而在 CRM+ERP 里,权限是天然存在的、业务因果是可验证的:Agent 说库存缺 48 台,我可以点进库存页数它;它下了一张采购单,首页的风险订单数必须从 2 变成 0。这让「Agent 到底干了什么」从一段自述变成了可当场核对的事实。
代价	要先造一整套自治的业务数据才能开始做 Agent,数据层花掉了约 1/5 的工作量。
5.2 为什么固定种子造假数据 + 显式埋雷	
决策	<code>generateSeed(42)</code> 确定性生成,再叠一层 <code>applyPlantedScenario</code> 。
理由	演示脚本必须每次跑出同一个结果。真随机意味着今天有风险订单、明天可能没有,面试当场跑出一句「未来两周没有风险」整个演示就废了。固定种子还带来一个额外好处:所有关键数字都能写进单元测试钉死——SKU-203 缺口 48 台、锐驰机电交期 7 天,实现改坏了测试会当场变红。
代价	数据层的断言变得脆:任何一处生成逻辑的微调都可能连带打翻好几条断言。这个代价真实兑现了(见 § 7.3 第 1 条)。
5.3 为什么保留权限分级,而没按质疑改成全员同数据	
决策	保留行级分权,但补上「显式边界」——每一个被过滤的视图都自报家门。
理由	收到的质疑是「老板看得到的、实际工作的人看不到,会有歧义」。这个质疑戳中了真问题,但指错了病根。行级分权在真实 B2B CRM 里是默认配置(Salesforce 的 OWD=Private + 角色层级),背后是佣金归属、防撬单、合规最小化三重理由,删掉它等于删掉这个项目最有辨识度的卖点。真正制造「歧义」的是静默过滤——页面上写着「订单」却只显示了一部分,且什么都不说。所以改的不是分权,是让每一次过滤都说话。
代价	如果本意就是要全员同数据,那这两期工作方向就错了。所以动手前先做了一次定向确认,而不是直接照做,也不是直接反驳。
5.4 为什么 CEO 从「零写权限」改成「有写权限 + 越权提示」	
决策	推翻初版设计,CEO 持有全部 4 个写工具,但执行本该由其他岗位负责的操作时,确认卡上追加一条越权代办提示,并在审计里标记 <code>override</code> 。
理由	「CEO 一个写操作都没有」是个想当然的设计——现实中没有任何一家公司会把 CEO 的写权限设为零。但直接放开又丢掉了「岗位分工」这个信息。「能做,但会被提示这不是你的活,而且留痕」比「不能做」更接近真实企业软件的做法。
代价	旧说法散落在四份文档和一处代码注释里需要同步更正,截至本报告仍有遗漏(见 § 7.2)。

5.5 为什么会话按角色隔离,还要支持归档

决策	会话列表只显示当前角色自己的;切角色时自动跳到该角色最近的会话;旧会话可归档收纳,归档后再提问会自动恢复。
理由	最初设计是「别人的会话能看见但只读」,收到的反馈是「会话应该只能看自己的记录」。这条反馈的价值在于:可见但不可用,是一种更糟的体验——既没提供价值,又制造了噪音。而且喂给模型的最近两轮如果跨角色串了,就是真实的越权泄漏(见 § 4.4)。
代价	跨角色回看能力消失。按「不静默过滤」原则补了一行「另有 N 个会话属于其他角色,切换到该角色后可见」。

六 Bad Case:7 个真实缺陷

按「现象 / 根因 / 处置 / 教训」写。**严重** = 会让核心卖点当场翻车;**中等** = 影响演示效果或数据自治。

6.1 · 招牌功能上的真实越权:传参数就能绕过权限

严重

现象	<code>simulate_delivery_risk</code> 不传参数时会走权限过滤,但一旦调用方显式传入订单号列表,就直接用、跳过过滤。这个工具四个角色都能调,于是销售代表只要传入任意订单号,就能读到不属于他的客户名、交期、缺口和风险等级。
根因	写代码时的思路是「默认路径要过滤」,没意识到「显式传参」是第二条路径。这类缺陷的共性是:安全检查写在了「取数据」那一步,而不是「返回数据」那一步。
处置	修法不是简单拒绝,而是求交集:传入的订单号与该角色可见范围取交集后再算;全部越权则返回明确原因,部分越权则返回允许的子集 + <code>deniedCount</code> 与 <code>deniedReason</code> ——既不静默丢弃,也不静默放行。补了两条负向测试钉死。
教训	这个洞正好打在项目对外宣称的头号卖点上。越是核心卖点,越要有人专门带着敌意去测它——自己写的代码自己找洞,成功率很低。这也是后来每次派发代码复审时,都把「这次最怕哪里出问题」明确写进指令的原因。

6.2 · 同类越权有四处,而修复清单只列了三处

严重

现象	应收账款数据有四个地方直接读全量、没有团队隔离。实测销售总监李娜看到的 5 笔逾期里有 3 笔属于其他团队的客户。
根因	同一个错误模式散落在四个文件里。而我在写修复任务书时只列了三处,漏了第四处。
处置	漏掉的这处被下一轮定向复审抓到。更糟的是:漏修让问题从「一致的错」变成了「显眼的矛盾」——修完之后,李娜首页的风险卡显示 5 笔,点开让 Agent 查却返回 2 笔,同一个屏幕上两个数字打架。破例追加一轮修复,并实测核对口径:李娜 2 笔 ¥161.4 万 = Agent 返回 2 笔,CEO 5 笔 ¥353.4 万 = Agent 返回 5 笔。
教训	两条。第一,发现一处同类缺陷时,第一反应应该是全局搜索这个模式,而不是修掉眼前这处——grep 一次 <code>db.receivables</code> 只要 5 秒。第二,修一半比不修更危险:不修是一个一致的错误,修一半是一个会被当场看见的矛盾。

6.3 · 切换角色的一瞬间,旧角色数据进了新角色上下文

严重

现象	一次问询要 6~40 秒。在这期间切换角色,旧角色那次问询完成后,会把答案写回已经被清空的会话历史。于是新角色的下一次提问,携带的上下文里带着旧角色看到的数据——这是真实的跨角色数据泄漏。
根因	典型的异步竞态:清空历史是同步的,写回历史发生在异步任务完成之后。而且角色切换器在问询进行中并没有被禁用,这条路径完全可达——更要命的是,演示脚本里就有一个「同一个问题切角色对照」的剧本,等于在主动触发它。
处置	在问询开始的那一刻把当前用户身份捕获成常量,整轮运行自始至终用这个身份;写回历史前校验「发起时的身份 == 当前身份」,不一致就丢弃。判断逻辑抽成纯函数并补了回归测试。同一个模式后来又消掉了另外两个出口(会话切换中的卡片串台、以及一个点了没反应的死按钮)。
教训	凡是「开始」和「结束」之间隔着几十秒的操作,都要假设期间什么都可能变。正确的模式是「开始时捕获,结束时校验」,而不是「结束时再去读当前值」。当同一类 bug 出现第三次时,要修的不是这一个 bug,是这个模式——出口的数量随功能增长,模式只有一个。

6.4 · 线上第一次真跑,演示高潮没出现

严重

现象	供应链主管问「未来两周要交付的订单有风险吗」,Agent 完整跑出 5 步计划、调了 5 次工具、数字全对。但计划的第 5 步「创建采购单」没有执行——模型走到「查供应商」就停下,改用文字写「建议向锐驰机电采购」。于是没有确认卡、没有人工批准、风险卡不消失、KPI 不归零。
根因	双重根因。(a) 执行提示词里对写工具的描述是被动的(「会弹确认卡,你正常调用即可」),而结尾的回答格式又要求「最后给 1-2 条可执行建议」——等于在指示模型用文字建议收尾。(b) 主循环把「本轮没返回工具调用」无条件当作任务完成,不检查计划里还有没有没执行的写步骤。
处置	改提示词措辞(现在规则 5 是「写操作必须真的执行…绝对不允许只在文字里写『建议采购 X 个』就收尾——那等于什么都没做」),并在主循环里加补推守卫(见 § 4.10)。5 条测试,全部以「模型被调用了几次」做断言。
教训	提示词里的每一句话都是在给模型下指令,包括那些你以为只是在描述格式的话。「最后给 1-2 条建议」本意是让回答更完整,实际效果是让模型认为「给建议」就是终点。更重要的一条:这个 bug 跑一百遍单元测试也发现不了,它只在接上真模型的第一次真实运行里暴露。任何依赖大模型行为的产品,都必须尽早做一次「接真模型、走完整流程、不 mock 任何东西」的验证。

6.5 · 追问一句就崩:Agent 完全没有记忆

严重

现象	上一轮 Agent 刚给出两条处理方案,用户接着说「采用处理方案 1」,回答是「需要更多信息来理解『处理方案 1』的具体内容」。再问「什么信息」,回答「用户问题过于模糊」。
根因	每一次提问都是独立单轮——每次都用系统提示词 + 新问题重新构造消息,没有携带任何对话历史。对模型来说,「采用处理方案 1」是一句凭空出现的话。
处置	加上「最近两轮」的历史,同时穿进规划和执行两个阶段(只加在执行阶段不够——规划器会先一步判定问题模糊而输出空计划)。规划器提示词里补了一条硬规则:「如果用户是在追问上文,结合上文还原成具体步骤,不要输出空 steps 要求用户澄清」。整套记忆设计见 § 4.4。
教训	这是典型的「功能清单齐了,但用户旅程断了」。设计时把「提问 → 规划 → 执行 → 回答」当成完整闭环,忘了真实用户的下一个动作是追问。做 Agent 产品,一定要把「用户会追问」当成第一类需求,而不是增强项。

6.6 · 错误文案在撒谎:并发限制被写成「网络不可用」

中等

现象	连续提问时反复出现「LLM 服务不可用,已切换录播模式」,而用户网络完全正常。
根因	免费档并发限制 2 路,一次 Agent 运行本身就是连续多次调用,再并行插一路就打满,返回 HTTP 429。而代码把「任何非 200」一律当作服务不可用,429 与真断网不可区分;面向用户的文案写的是「当前网络不可用」——这是一句假话。
处置	给 429 单独定义了一个异常类型 <code>LlmRateLimited</code> (继承自 <code>LlmUnavailable</code> ,这样上层的兜底逻辑不用改),退避 6 秒重试一次,文案改为「并发已满 · 免费档并发上限为 2 路」。
教训	错误文案是产品的一部分,写错了就是在向用户撒谎。「网络不可用」会让用户去检查 WiFi、刷新页面、怀疑自己,而真正该做的是等 6 秒。把不同的失败原因映射成同一句话,等于放弃了这个产品的可诊断性。

6.7 · 首页 KPI 和演示高潮零联动

中等

现象	首页第三张 KPI 卡设计为「库存告警 SKU 数」,验收标准写着「Agent 批准采购单后该 KPI 减 1」。但这个数字 永远不可能变 ——埋雷的那个 SKU 可用量本来就高于安全库存,根本不计入告警,批准采购后更不会。
根因	指标口径是拍脑袋定的,定的时候没有拿真实数据验算过它会不会动。
处置	把口径改成「交期风险订单数」——它正是演示高潮所依赖的那个数字,批准采购单后从 2 变成 0,卡片同步消失。 把一个死指标换成了整个演示的验收锚点。
教训	看板上的每一个数字都应该回答「它什么时候会变、变了说明什么」。 不会变的数字不配占据首屏。这条对做 BI / 看板类产品通用:先想清楚指标的 变化 有什么意义,再决定要不要放。

七 做得不好的地方

这一节尽量写得难看一点。能被追问倒的地方,自己先说出来。

7.1 工程欠账

1 Agent 上下文没有任何压缩机制

§ 4.3 已经说过: `messages` 只增不减,靠工具自觉返回精简结构。这是整个 Agent 设计里最不能规模化的一处。演示规模下够用,但它是一个约定而不是机制,任何一个新加的工具只要返回大结果,就能把上下文撑爆。

2 界面逻辑没有自动化测试

141 条测试全是纯函数级,没有引入 `jsdom` 或浏览器测试环境,所有交互逻辑都靠人工点。代价是真实的:为了让逻辑可测,我不得不反复把判断从组件里抽成纯函数——这有好处(逻辑更清晰),但也有一个已知盲点:纯函数测试锁的是判断条件,不是调用点。参数写反了,测试照样全绿。目前靠人工核对参数顺序兜住,这不可持续。

3 审计日志有数据、没有界面

每一次工具调用、每一次批准与拒绝都被 `pushAudit` 记录了下来,结构完整(时间 / 角色 / 用户 / 工具 / 参数 / 耗时 / 是否越权),保留最近 200 条。但界面上没有任何入口能看到它。这是一个明确承诺过却没交付的功能,推迟理由是「临近演示不加新页面」——理由成立,欠账是欠账。而且它本可以直接回答「Agent 到底干了什么」这个必问题。

4 动态重规划靠正则匹配自然语言

见 § 4.10。模型换个说法就匹配不上。知道更好的做法(做成 `amend_plan` 工具),当时没有时间设计防滥用约束。

5 移动端完全没做

用手机打开是三栏桌面布局硬挤在小屏上。当初测手机只是为了验证国内网络能否访问,不是为了移动端体验。这是一个有意识的范围外,但如果对方拿手机点开,观感就是差。

7.2 产品欠账

1 文档落后于代码,而且是可验证的落后

写这份报告时逐条核对源码,当场查出两处漂移:

- `src/agent/loop.ts:59` 的注释还写着「CEO 没有写工具」,这是初版说法,现在 CEO 持有全部 4 个写工具(行为正确,注释错误);
- PRD 里各角色工具数写的是 9/11/8/11,实测是 9/11/10/15。

文档一旦落后于代码,它的每一句话都需要被重新验证才敢引用,价值大打折扣。

2 项目门面上一张截图都没有

代码仓库首页至今没有任何界面截图。原因是不想放指向不存在图片的链接(那会显示成破图,比没有更糟),所以留了占位和拍摄清单。理由成立,但一个视觉产品的门面上没有视觉,这是硬伤。

3 降级体验不干净

模型在执行中途不可用时,录播会从中途接管,真实卡片和重放卡片叠在同一条会话里,出现两套计划清单。已知、未解决。

7.3 我个人的判断错误(三条)

错误	怎么被发现的	教训
改了客户归属,忘了带动订单归属,造成 160 张订单里 53 张的归属人与其客户的归属人对不上—— 33% 的数据自相矛盾 。张伟名下 25 张订单里 21 张的客户根本不归他	我自己写探针实测发现。此前的代码复审把它描述为「取决于随机数、可能不发生」,实测证明它已经 100% 发生了	数据生成是有顺序的, 改上游必须检查下游 。做决定的人要为决定的连带后果负责,而不是等实现方发现。另外:「 可能不发生 」这种措辞出现时,去测一下
把并发限制误判成「 免费额度耗尽 」	被要求去平台查证后纠正	两者处置方式完全不同——额度耗尽只能等第二天,并发冲突退避几秒就好。 没实测就不要下定性结论,哪怕它听起来很合理
写修复清单时漏列了第四处同类越权(即 § 6.2)	下一轮定向复审抓到	发现一处同类缺陷, 先全局搜模式,再动手 。漏修让一致的错误变成了屏幕上可见的矛盾
补充一句方法层面的:这个项目的开发流程是「写任务书 → 派发实现 → 独立代码复审 → 逐条裁决」,实现方和复审方互不知情。§ 6.1、§ 6.2、§ 6.3 这三个缺陷 全部是独立复审抓出来的,不是我自己发现的 。		

八 改进方向

给优先级判断,不是罗列。

8.1 Agent 线(最优先)

优先	做什么	怎么做
P0	上下文压缩策略	三层递进:(a) 工具返回值超过阈值时截断并附「已省略 N 条」;(b) 执行环轮次超过 6 轮时,把前面的工具结果压成一段摘要消息;(c) 记录每轮的 prompt token,超阈值主动降级。 当前一个都没有 ,这是 § 7.1 第 1 条的直接解
P0	成本护栏	目前 token 用量只是展示出来, 没有任何上限 。一个进入异常循环的 Agent 可以几分钟烧完免费额度。至少要有单次问询的 token 上限,超限时中止并明确告知
P1	工具调用重试策略	现在工具执行抛异常就把错误字符串回灌给模型,由模型自己决定怎么办。应该区分「参数错了(可重试,给模型明确的纠正提示)」和「数据不存在(不可重试)」
P1	重规划走结构化通道	把正则匹配 需要追加步骤: 换成 amend_plan 工具,并加约束(单次运行最多追加 2 步)
P2	可解释性回补	思考链关闭换来了延迟,但丢了推理轨迹。可以改成保留 reasoning_content 但默认折叠、按需展开——延迟问题用流式输出解
P2	什么时候引入 RAG	见 § 4.5:只有出现合同条款、SOP 制度、历史工单这类非结构化知识时才引入。 且必须先解决检索层的权限前置过滤 ,不能检索完再筛

8.2 产品线

优先	做什么	为什么是这个优先级
P0	补齐文档与代码的一致性	一处过时说法被当场戳穿,整份文档可信度归零。成本几小时,收益是不翻车
P0	补三张真实截图进门面	视觉产品没有视觉,是转化率问题
P1	把审计日志做出界面	已承诺、数据已在、只差一个页面,而且它天然回答「Agent 干了什么」这个必问题
P1	会话搜索与导出	归档解决了「太多」,没解决「找不到」
P2	移动端适配	范围明确在外。要做就是把右侧抽屉改成底部弹层,是一期独立工作,不能顺手塞

8.3 工程线

- 引入浏览器级测试,优先覆盖三条链路——确认卡的批准/拒绝、角色切换期间的并发问询、会话切换与归档。这三条都是出过真实缺陷的地方,也正是纯函数测试的盲区。**不需要全覆盖,只覆盖出过事的地方,性价比最高。**
- 把「显式边界」抽象成统一机制——现在是每个页面各自调用文案函数,新加页面容易忘(采购页就漏过一次)。应该做成数据层返回边界信息、界面层强制消费,忘了就编译不过。
- 降级要么在开始时决定,要么就不降级——解决执行中途切录播导致两套卡片并存的问题。

九 可复用的经验清单

如果这份报告只留一页给下一个项目,就是这一页。

做 Agent 架构(6 条)

- 1 **先想清楚检索的性质,再决定要不要上 RAG**
filter+aggregate 用结构化查询,语义定位才用向量。用错了不只是浪费,还会在精确数字上返回近似匹配。
- 2 **RAG 的难点不是召回质量,是权限**
权限过滤必须在检索层前置,不能召回后再筛。一处漏标就是静默泄漏,没有编译器能帮你。
- 3 **把「用户会追问」当第一类需求**
不是增强项。没有对话历史的 Agent,第二句话就会崩。而且历史要同时穿进规划和执行两个阶段。
- 4 **提示词里的每一句都是指令,包括你以为只是格式说明的那句**
「最后给 1-2 条建议」让模型认为给建议就是终点,于是它不再调用工具。
- 5 **写操作必须有手工确认,而且拒绝路径必须是真的**
拒绝之后不能换个说法再弹一次,否则「你有否决权」是空话。
- 6 **必须尽早接真模型跑一次完整流程,一次都不 mock**
单元测试跑一百遍也发现不了模型「不肯调工具」这类问题。

做权限与数据边界(5 条)

- 1 **不要静默过滤**
少给用户看了东西,就当场说清少了什么、为什么少。这是全项目最值钱的一条。
- 2 **区分「范围外还有」和「整类无权」**
两者文案完全不同,混用会出现「你无权查看内没有记录」这种拗口且错误的表达。
- 3 **权限检查要写在「返回数据」那一步,不是「取数据」那一步**
否则多一条显式传参的路径,就多一个洞。
- 4 **数据层不该依赖工具层的门禁配置**
即使当前 allowedRoles 保证了等价性,也要走 scope——改配置时数据边界要自动跟上。
- 5 **发现一处同类缺陷,先全局搜模式**
修一半比不修更危险:不修是一致的错,修一半是当场可见的矛盾。

做状态与异步(2 条)

- 1 **开始时捕获,结束时校验**
凡是开始和结束之间隔着几十秒的操作,期间什么都可能变。不要在结束时才去读当前值。
- 2 **同一类 bug 出现第三次时,修的是模式不是 bug**
出口的数量随功能增长,模式只有一个。

做技术选型(2 条)

- 1 **核心卖点所在的那一层,不要用框架**
这个项目的卖点是「权限如何贯穿每一次工具调用」,那就必须是自己能逐行解释的 157 行代码,而不是框架里一层不透明的抽象。其余的地方尽量用现成的。

2 每一个「没用什么」都要说得有理由

没上 UI 组件库、没上 React Query、没上 Agent 框架,各有各的理由。选型的说服力一半来自你选了什么,一半来自你能解释清楚你放弃了什么。

最后一条,也是我认为最重要的一条: 这个项目里所有真正严重的缺陷——两处越权、一处跨角色泄漏、一处演示高潮不出现——**没有一个是设计阶段被想到的**。它们全部来自两个地方:独立的代码复审,和接上真实模型的实盘运行。再周密的设计也替代不了验证。**与其把时间全花在把方案想得更完美,不如留出一半时间,去设计怎么尽早、尽狠地验证它。**